

Creator DNA OS

Creator DNA OS is a production-ready AI creator platform designed to empower content creators with intelligent tools for generating, managing, and exporting various forms of content. This platform leverages a modular microservices architecture, built with Python FastAPI for the backend, React and TypeScript for the frontend, and orchestrated with Docker and GitHub Actions for seamless development and deployment.

Table of Contents

1. [Features](#)
2. [Tech Stack](#)
3. [Architecture](#)
4. [Folder Structure](#)
5. [Getting Started](#)
 - [Prerequisites](#)
 - [Installation](#)
 - [Environment Variables](#)
6. [Development](#)
 - [Running Services Locally](#)
 - [Backend Development](#)
 - [Frontend Development](#)
7. [Testing](#)
8. [Deployment](#)
9. [API Documentation](#)
10. [Security](#)
11. [Logging & Error Handling](#)

Features

Creator DNA OS provides a comprehensive suite of features to streamline the content creation workflow:

- **Authentication:** Secure user registration, login, and profile management using JWT.

- **Dashboard:** Personalized overview of recent projects, Creator DNA profile insights, and quick access to AI tools.
- **Projects:** Full CRUD (Create, Read, Update, Delete) functionality for managing content projects, including search capabilities.
- **Creator DNA:** Define and refine writing preferences, prompt templates, brand settings, and workflow preferences. The system learns from user edits to continuously improve AI outputs.
- **AI Services:** Modular AI capabilities for generating original scripts, titles, descriptions, hashtags, summaries, and brainstorming ideas.
- **Media Management:** Import YouTube URLs, extract transcripts, generate subtitles, and export various media formats.
- **Export:** Export generated content to Markdown, PDF, DOCX, and TXT formats.
- **API:** A robust REST API with endpoints for every service, enabling seamless integration and extensibility.

Tech Stack

Backend

- **Python:** Primary language for all backend services.
- **FastAPI:** High-performance web framework for building APIs.
- **SQLAlchemy:** ORM for interacting with the PostgreSQL database.
- **PostgreSQL:** Relational database for persistent storage.
- **JWT Authentication:** Token-based authentication for secure API access.
- **Redis:** Used for caching and rate limiting.

Frontend

- **React:** JavaScript library for building user interfaces.
- **TypeScript:** Statically typed superset of JavaScript for enhanced code quality.
- **Tailwind CSS:** Utility-first CSS framework for rapid UI development.

Infrastructure

- **Docker:** Containerization for consistent development and deployment environments.
- **GitHub Actions:** CI/CD pipeline for automated testing, building, and deployment.

- **REST API:** Standardized communication protocol between frontend and backend services.

AI

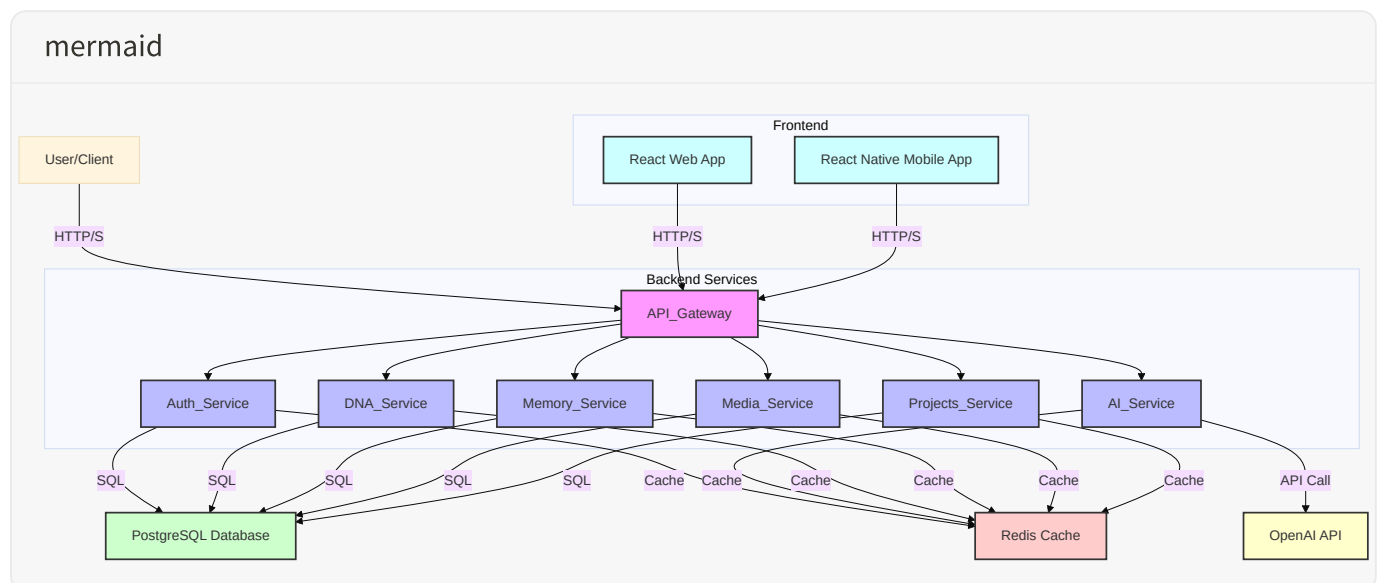
- **OpenAI API:** Integration with OpenAI's powerful language models for content generation.
- **Modular AI Services:** Designed for easy integration of additional AI models and capabilities.

Video

- **FFmpeg:** Command-line tool for handling multimedia data, used in media processing.

Architecture

Creator DNA OS follows a **microservices architecture** pattern, where the application is broken down into smaller, independent services. Each service is responsible for a specific business capability and communicates with others via a central API Gateway.



Key Architectural Components:

- **API Gateway:** A central entry point for all client requests, routing them to the appropriate backend service. It handles cross-cutting concerns like authentication, rate limiting, and logging.
- **Microservices:** Independent, loosely coupled services (Auth, AI, DNA, Memory, Media, Projects) that can be developed, deployed, and scaled independently.
- **Database:** A single PostgreSQL instance is used for simplicity in this scaffold, but in a production environment, each microservice might have its own dedicated database.

- **Redis:** Used for caching frequently accessed data and for rate limiting to protect services from abuse.
- **Frontend Applications:** Separate React web and React Native mobile applications consume the API Gateway.

Folder Structure

Plain Text

```
creator-dna-os/
├── apps/
│   ├── web/           # React/TypeScript web application
│   └── mobile/        # React Native mobile application (placeholder)
├── services/
│   ├── ai/           # AI content generation service
│   ├── dna/          # Creator DNA profile and preferences service
│   ├── memory/       # User memory and learning service (placeholder)
│   ├── media/        # Media processing (YouTube, transcripts, export) serv
│   ├── auth/         # User authentication and authorization service
│   ├── projects/     # Project management service
│   └── gateway/      # API Gateway for routing and security
├── packages/
│   ├── ui/           # Shared UI components (placeholder)
│   └── shared/        # Shared Python utilities (config, database, security,
├── database/
│   ├── migrations/   # Alembic database migrations
│   └── seeds/        # Database seed data
├── docs/             # Project documentation
├── scripts/          # Utility scripts
├── tests/
│   ├── unit/         # Unit tests for individual components
│   ├── integration/  # Integration tests for service interactions
│   └── api/          # API tests for endpoints
├── .github/
│   └── workflows/
│       └── ci.yml     # GitHub Actions CI/CD pipeline
├── .env.example       # Example environment variables
├── docker-compose.yml # Docker Compose configuration for all services
├── docker-compose.dev.yml # Docker Compose overrides for local development
└── README.md         # This file
```

Getting Started

Prerequisites

Before you begin, ensure you have the following installed:

- **Git:** For cloning the repository.
- **Docker & Docker Compose:** For running the application in containers.
- **Node.js & npm/yarn:** For frontend development (if not using Docker for frontend).
- **Python 3.11+ & pip:** For backend development (if not using Docker for backend).

Installation

1. Clone the repository:

Bash

```
git clone https://github.com/your-username/creator-dna-os.git
cd creator-dna-os
```

2. Copy environment variables:

Bash

```
cp .env.example .env
```

Edit the `.env` file and fill in the required values, especially `SECRET_KEY` and `OPENAI_API_KEY`.

3. Build and run the services (development mode):

Bash

```
docker compose -f docker-compose.yml -f docker-compose.dev.yml up --build
```

This command will build all Docker images, set up the PostgreSQL and Redis containers, and start all backend and frontend services with hot-reloading enabled for development.

For production-like deployment (without hot-reloading and development volumes):

Bash

```
docker compose up --build -d
```

Environment Variables

The `.env` file configures various aspects of the application. Here are the key variables:

Variable	Description	Default Value
<code>APP_NAME</code>	Name of the application.	<code>Creator DNA OS</code>
<code>APP_ENV</code>	Application environment (<code>development</code> , <code>staging</code> , <code>production</code>).	<code>development</code>
<code>DEBUG</code>	Enable/disable debug mode.	<code>true</code>
<code>LOG_LEVEL</code>	Logging level (<code>INFO</code> , <code>DEBUG</code> , <code>WARNING</code> , <code>ERROR</code> , <code>CRITICAL</code>).	<code>INFO</code>
<code>SECRET_KEY</code>	CRITICAL: Used for JWT signing. MUST be a strong, random string.	<code>change-me-to-a-long-random-string-in-production</code>
<code>JWT_ALGORITHM</code>	Algorithm for JWT.	<code>HS256</code>
<code>ACCESS_TOKEN_EXPIRE_MINUTES</code>	Expiration time for access tokens in minutes.	<code>60</code>
<code>REFRESH_TOKEN_EXPIRE_DAYS</code>	Expiration time for refresh tokens in days.	<code>30</code>
<code>POSTGRES_USER</code>	PostgreSQL username.	<code>cdna_user</code>
<code>POSTGRES_PASSWORD</code>	PostgreSQL password.	<code>cdna_secret</code>
<code>POSTGRES_DB</code>	PostgreSQL database name.	<code>creator_dna</code>
<code>DATABASE_URL</code>	Full PostgreSQL connection string.	<code>postgresql://cdna_user:cdna_secret@localhost:5432/creator_dna</code>
<code>REDIS_URL</code>	Redis connection string.	<code>redis://localhost:6379/0</code>
<code>OPENAI_API_KEY</code>	CRITICAL: Your OpenAI API key.	<code>sk-your-openai-api-key-here</code>
<code>OPENAI_MODEL</code>	OpenAI model to use (e.g., <code>gpt-4o</code>).	<code>gpt-4o</code>
<code>OPENAI_MAX_TOKENS</code>	Maximum tokens for OpenAI responses.	<code>4096</code>
<code>OPENAI_TEMPERATURE</code>	OpenAI temperature setting.	<code>0.7</code>
<code>RATE_LIMIT_PER_MINUTE</code>	Global API rate limit.	<code>60</code>
<code>RATE_LIMIT_AI_PER_MINUTE</code>	AI service specific rate limit.	<code>10</code>

UPLOAD_DIR	Directory for media uploads.	./uploads
MAX_UPLOAD_SIZE_MB	Maximum allowed media upload size.	100
ALLOWED_MEDIA_TYPES	Comma-separated list of allowed media MIME types.	video/mp4,audio/mpeg,audio/wav
YOUTUBE_API_KEY	YouTube Data API v3 key for transcript extraction.	your-youtube-data-api-v3-key
CORS_ORIGINS	Comma-separated list of allowed CORS origins.	http://localhost:3000,http://localhost:5173
VITE_API_BASE_URL	Frontend: Base URL for the API Gateway.	http://localhost:8000
VITE_APP_NAME	Frontend: Application name displayed in UI.	Creator DNA OS
AUTH_SERVICE_URL	Internal URL for Auth service (used by API Gateway).	http://localhost:8001
AI_SERVICE_URL	Internal URL for AI service (used by API Gateway).	http://localhost:8002
DNA_SERVICE_URL	Internal URL for DNA service (used by API Gateway).	http://localhost:8003
MEMORY_SERVICE_URL	Internal URL for Memory service (used by API Gateway).	http://localhost:8004
MEDIA_SERVICE_URL	Internal URL for Media service (used by API Gateway).	http://localhost:8005
PROJECTS_SERVICE_URL	Internal URL for Projects service (used by API Gateway).	http://localhost:8006

Development

Running Services Locally

To run the services locally with hot-reloading and development-specific configurations, use the `docker-compose.dev.yml` override file:

```
Bash
```

```
docker compose -f docker-compose.yml -f docker-compose.dev.yml up --build
```

This command will:

- Start PostgreSQL and Redis.
- Build and run all backend services (Auth, AI, DNA, Memory, Media, Projects, Gateway) with `uvicorn --reload` .
- Build and run the frontend web application with `npm run dev` .

The frontend will be accessible at `http://localhost:3000` and the API Gateway at `http://localhost:8000` .

Backend Development

Each backend service resides in its own directory under `services/` . To work on a specific service:

1. Ensure Docker Compose is running in development mode.
2. The `docker-compose.dev.yml` mounts the local service directories into the containers, so changes to your code will trigger hot-reloads.
3. Install Python dependencies locally if you prefer to run tests or linters outside Docker:

Bash

```
pip install -r services/requirements.txt
pip install -e packages/shared # Install shared package in editable mode
```

Frontend Development

The web frontend is located in `apps/web/` .

1. Ensure Docker Compose is running in development mode.
2. The `docker-compose.dev.yml` mounts the local `apps/web` directory into the container.
3. You can also run the frontend independently if needed:

Bash

```
cd apps/web
npm install
npm run dev
```

The frontend will be available at `http://localhost:3000` .

Testing

Tests are located in the `tests/` directory, organized by type (unit, integration, api).

To run backend tests (requires `pytest` and `pytest-asyncio`):

Bash

```
pytest tests/
```

To run frontend tests (requires `vitest`):

Bash

```
cd apps/web  
npm test
```

Deployment

The project is configured for continuous integration and continuous deployment (CI/CD) using GitHub Actions. The `ci.yml` workflow handles:

- **Linting & Type-checking:** Ensures code quality for both backend (Python) and frontend (TypeScript).
- **Testing:** Runs unit and integration tests for all services.
- **Build & Push Docker Images:** On `main` branch pushes, Docker images for each service are built and pushed to GitHub Container Registry (GHCR).
- **Deployment:** Automatically deploys the latest images to a production server via SSH. This step requires `DEPLOY_HOST`, `DEPLOY_USER`, and `DEPLOY_SSH_KEY` to be configured as GitHub Secrets in your repository.

To deploy manually:

1. Ensure your `.env` file is configured for production.
2. Build and push Docker images:

Bash

```
docker compose build  
# Tag and push images to your registry (e.g., Docker Hub, GHCR)  
# Example: docker tag creator-dna-os-auth ghcr.io/your-username/creator-dna-  
# docker push ghcr.io/your-username/creator-dna-os-auth:latest
```

3. On your production server, pull the latest images and restart services:

Bash

```
cd /path/to/creator-dna-os
docker compose pull
docker compose up -d --remove-orphans
```

API Documentation

Each FastAPI service automatically generates OpenAPI (Swagger) documentation. Once the services are running, you can access the documentation:

- **API Gateway:** `http://localhost:8000/docs`
- **Auth Service:** `http://localhost:8001/docs`
- **AI Service:** `http://localhost:8002/docs`
- ...and so on for other services.

Security

Security is a paramount concern for Creator DNA OS. Key security measures include:

- **JWT Authentication:** Securely verifies user identity for API access.
- **Password Hashing:** All user passwords are hashed using `bcrypt` before storage.
- **Rate Limiting:** Implemented using Redis to prevent abuse and protect services.
- **Input Validation:** Pydantic models are used extensively to validate all incoming API requests.
- **Environment Variables:** Sensitive information like API keys and database credentials are managed via environment variables and not hardcoded.
- **CORS:** Configured to allow requests only from specified origins.

Logging & Error Handling

- **Structured Logging:** All services use a consistent logging configuration, outputting structured logs to `stdout` for easy collection by log management systems.
- **Centralized Error Handling:** Custom exception classes and FastAPI's `HTTPException` are used to provide consistent and informative error responses across all APIs.
- **Configuration:** Logging levels can be adjusted via the `LOG_LEVEL` environment variable.